SCRAM

Product Requirements Document: Governance control for action-taking AI agents

Feature name	SCRAM — HackSprint MVP
Co-owners, front end (UI/UX)	Natalie Walker, Nicole Rodriguez
Co-owners, back end	Jason Zeng, Aisling Leiva-Davila
Date	October 15, 2026

1. PROBLEM

An AI agent can have the technical ability to act without having the authority to act. In high-consequence settings, the same model that proposes an action is often trusted to judge whether it is allowed, and logs are reviewed only after the action has happened.

Problem statement. Teams testing action-taking AI agents need proof that each action stayed within granted authority and that a bad result was recovered, because today an LLM can propose, self-approve and execute with nothing stopping it but a log.

What SCRAM is. A developer-facing safety harness. It checks a proposed action against a governing rule, enforces GREEN / YELLOW / RED outside the model, binds human approval to the exact action, executes in a sandbox, verifies the result, rolls back on failure, and records the whole trajectory for evaluation.

1a. Background & Dependencies

- **Event.** Daytona HackSprint #6, New York, October 16, 2026. Hacking 10:00 AM to 3:30 PM submission; finalists present in 3 minutes.
- **Scenario.** A fictional Nuclear Commission rulebook and a simulated reactor. SCRAM is named for the emergency shutdown principle. Nothing connects to a real facility, and no certification is claimed.
- **Context.** The NRC is reducing inspection hours and moving toward more industry self-reporting, a change that is publicly disputed. Less outside inspection raises the value of systems that produce their own proof.
- **Locked stack.** React + TypeScript + Vite front end with custom CSS; Python + FastAPI back end; REST between them; Gemini API (free tier) as the model. Details in the SCRAM Build Spec.

Dependency	Role in SCRAM	If unavailable
Daytona	Sandbox that runs the approved action	Local executor, same interface
Arize AX	Ordered trace and evaluators	Cached trace, rendered in-app
SurrealDB	Rulebook, approvals, audit history	Local JSON fixture
ElevenLabs	Control-room voice: reads the exact action before approval, announces RED and rollback	Pre-generated audio clips
Gemini API (free tier)	Parses the request; writes explanations only	Prepared typed actions
Anchor Browser	Governed browser action, only if it has a real job	Dropped
AWS Bedrock (free event	Optional swap for Gemini if enabled in the first 30 minutes	Stay on Gemini

sandbox)

No tool costs money. No sponsor tool is required by the rules; each is used because it does a real job and is judged for a Best Use prize.

1b. Assumptions

- Teams moving agents toward production want evidence that authorization and approval were actually followed, not just logged. This is a hypothesis; no interviews have been done.
- The category is crowded (AWS AgentCore Policy, Microsoft Entra Agent ID, Open Policy Agent, governance suites). SCRAM claims only the combined, reproducible safety case.
- The first buyer is an AI platform or security lead testing agents before production. The nuclear scenario is a stress test, not the first market.
- Daytona snapshot support is unconfirmed. Until confirmed, a checkpoint is a stored copy of the simulated state.
- **Falsification.** If target users say their policy and observability stack already proves end-to-end authorization and recovery, the niche must narrow further.

2. SUCCESS METRICS

Metric	Target
RED or invalid-approval actions that reach the executor	0
Policy state matches the truth set across the five cases	5 of 5
YELLOW executes only with a valid, single-use, hash-matched approval	100%
Seeded bad result rolled back and restoration verified	100%
Ordered Arize trace with four evaluators passing	At least 1 live run
Primary demo (YELLOW, fault, rollback)	Under 90 seconds
Reset to the deterministic starting state	100%

3. REQUIREMENTS

P0 and P1 mark build order, not undecided scope. Everything listed ships if time allows; P0 ships first.

User Journey 1: Submit a request

Enter and normalize

- [P0] Operator types a request or loads one of five prepared cases.
- [P0] Gemini converts it into a typed action: tool, target, parameters, expected postcondition.
- [P0] Operator sees the original request and the normalized action side by side.
- [P0] A request that cannot be parsed, or parses with low confidence, is blocked.

Fingerprint

- [P0] SCRAM hashes the normalized action (SHA-256 over canonical JSON).

User Journey 2: Establish authority

Find the rule

- [P0] SCRAM retrieves the governing rule from the six-rule synthetic rulebook.
- [P0] Operator sees rule ID, excerpt and policy version.
- [P0] No applicable rule means RED under NC-06.
- [P1] Conflicting rules are shown, never silently resolved.

Decide

- [P0] Deterministic Python policy returns GREEN, YELLOW or RED with a reason code.
- [P0] The model cannot set, change or bypass the policy state.
- [P0] RED never reaches the executor, and the voice announces "SCRAM activated."
- [P0] If the policy service or audit log is unavailable, nothing executes.

User Journey 3: Authorize the exact action

Approve

- [P0] YELLOW pauses before execution and shows exactly what will run.
- [P0] The voice reads the exact action aloud before the operator decides ("Approval requested: increase output two percent, to seventy-four").
- [P0] Operator approves or rejects that exact action.
- [P0] Approval is single-use, expires in five minutes, and is bound to the approver and the action hash.

Protect the approval

- [P0] The executor rechecks the hash immediately before changing state.
- [P0] A changed parameter, expired approval or reused approval is blocked with its reason.

User Journey 4: Execute, verify, recover

Execute

- [P0] SCRAM checkpoints the state before any write.
- [P0] The sandbox executor runs only the approved call.
- [P0] A visible "Inject executor fault" switch makes the tool apply +20% instead of +2%.

Verify and recover

- [P0] A deterministic verifier compares the actual state with the expected postcondition.
- [P0] On failure, SCRAM restores the checkpoint, re-reads the state and confirms the checksum matches.
- [P0] If rollback fails, SCRAM locks all writes until a human resets it.
- [P0] The voice says "Verification failed. Safe state restored."
- [P0] Every voice line has a pre-generated clip, so the demo never waits on the network.

User Journey 5: Prove what happened

Trace

- [P0] Every step is a span in the required order: normalize, policy, approval, checkpoint, execute, verify, rollback, audit.
- [P0] The trace is shown inside SCRAM, not only in Arize.
- [P1] One click opens the run in Arize.

Evaluate

- [P0] Four evaluators run: unauthorized execution, approval integrity, rollback verification, trajectory order.
- [P0] An unsafe baseline and SCRAM run the same five cases side by side.

User Journey 6: Reset

- [P0] One click restores the rulebook, cases and simulated state to their starting values.
- [P0] The demo runs repeatedly without editing any file.

4. APPENDIX

Design Decisions

Decision	Rationale
Policy is deterministic code outside the model	An agent must not authorize itself
Approval is bound to a hash of the exact action	Approval cannot be reused or quietly broadened
Checkpoint is a stored state copy unless Daytona snapshots are confirmed	No dependency on an unverified feature; rollback is proven by re-reading state
The fault-injection switch is visible	Judges see the failure is deliberate; the clean path can run first
Positioned as a safety harness, not a governance suite	The broad category is crowded; the combined proof is the wedge
React, Vite, FastAPI and custom CSS	No routing or SSR needed; Python has the strongest Daytona and Arize support; no styling setup time
Anchor ships only if it has a real job by 1:30 PM	A logo-only integration counts against us
ElevenLabs is P0 and speaks at each decision	The voice is part of the safety story, and it competes for the ElevenLabs prize
Gemini free tier is the model; Bedrock is an optional swap	Costs nothing and works tonight; the model's job is small because policy is our own code
Visual identity: the Codex (iron-gall ink, red chalk, construction lines)	Reads as a trusted instrument; distinct from REDLINE

Open Questions

- Does Daytona support snapshot and restore? Confirm tonight; otherwise use state copies.
- Proposed split: Jason and Natalie lead SCRAM; Aisling and Nicole lead REDLINE. Confirm tonight.
- Does the free AWS event sandbox include Bedrock? Test in the first 30 minutes; if not, stay on Gemini.

Other Links

- SCRAM Build Spec (SCRAM_BUILD_SPEC.md) — binding for the coding agent
- SCRAM & REDLINE Storyboards canvas
- Daytona HackSprint #6 event brief
- NRC proposed rule, Docket NRC-2025-1138 (context only)